БОЛЬШОЙ БАНК ВОПРОСОВ К КОЛЛОКВИУМУ

Алгоритмы и структуры данных • 1 семестр • ИТМО М3102

Блиц-теормин (50) • Полные билеты (25) • Ловушки экзаменаторов (25) • Командные баттлы (15)

ЧАСТЬ 1. БЛИЦ-РАУНД: ТЕОРМИН (50 ВОПРОСОВ ДЛЯ 1-МИНУТНОГО ОТВЕТА)

1. Дайте строгое определение $O(g(n))$.

Ответ: $f(n) \in O(g(n))$, если существуют константы $c > 0$ и $n_0 \in \mathbb{N}$, такие что для всех $n \geq n_0$ выполняется: $0 \leq f(n) \leq c \cdot g(n)$.

2. Дайте строгое определение $\Omega(g(n))$.

Ответ: $f(n) \in \Omega(g(n))$, если существуют константы $c > 0$ и $n_0 \in \mathbb{N}$, такие что для всех $n \geq n_0$ выполняется: $f(n) \geq c \cdot g(n) \geq 0$.

3. Дайте строгое определение $\Theta(g(n))$.

Ответ: $f(n) \in \Theta(g(n))$, если $f(n) \in O(g(n))$ и одновременно $f(n) \in \Omega(g(n))$. То есть существуют $c_1, c_2 > 0$ и n_0, такие что $c_1 g(n) \leq f(n) \leq c_2 g(n)$ при всех $n \geq n_0$.

4. В чём разница между $O(g(n))$ и $o(g(n))$?

Ответ: O — нестрогая верхняя граница ($\limsup \leq c$), а o — строго меньший порядок роста: $\lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$. Например, $2n \in O(n)$, но $2n \notin o(n)$, зато $n \in o(n^2)$.

5. Что такое модель word RAM и каков размер слова?

Ответ: Модель вычислений, где память состоит из ячеек (машинных слов) размером $w \geq \log_2 n$ бит. Базовые арифметические, побитовые операции и чтение/запись выполняются за $O(1)$ такт.

6. Что принято считать атомарной операцией в алгоритмах?

Ответ: Элементарную операцию процессора, занимающую константное время: сложение двух чисел, сравнение, присваивание, разыменование одного указателя.

7. Что такое дополнительная память (Auxiliary Space)?

Ответ: Объём памяти, выделяемый алгоритмом **сверх** входных и выходных данных для внутренних рабочих структур (стек, переменные, буферы).

8. Каков расход памяти рекурсивной функции с максимальной глубиной вызовов h?

Ответ: $O(h)$ стековой памяти, так как в стеке одновременно хранится не более h кадров активации (stack frames).

9. Что такое In-place алгоритм?

Ответ: Алгоритм, требующий $O(1)$ дополнительной памяти (для быстрой сортировки допускается $O(\log n)$ стековой памяти) и модифицирующий входные данные на месте.

10. Что такое устойчивость (stability) сортировки?

Ответ: Свойство сохранять исходный относительный порядок следования элементов с одинаковыми ключами.

11. В чём ключевое преимущество Insertion Sort перед QuickSort?

Ответ: Сортировка вставками адаптивна (работает за $O(n)$ на почти отсортированных данных), устойчива, in-place и имеет минимальную константу на малых $n \leq 16$.

12. Каково время работы Insertion Sort в лучшем случае?

Ответ: $O(n)$ — когда массив уже отсортирован (внутренний цикл завершается за 1 сравнение на каждом шаге).

13. Какова точная связь времени Insertion Sort с инверсиями?

Ответ: Число сравнений и перемещений равно $I + O(n)$, где I — точное число инверсий в массиве.

14. Каково время работы MergeSort во всех трёх случаях?

Ответ: Строго $\Theta(n \log n)$ всегда: в лучшем, среднем и худшем случаях, независимо от порядка данных.

15. Сколько дополнительной памяти требует MergeSort на массиве?

Ответ: $O(n)$ памяти под временный буфер слияния (плюс $O(\log n)$ на стек вызовов).

16. Можно ли реализовать MergeSort на связном списке за $O(1)$ памяти?

Ответ: Да, так как слияние списков выполняется только перенаправлением указателей без выделения новых узлов.

17. Как подсчитать число инверсий при слиянии подмассивов в MergeSort?

Ответ: Когда берется элемент из правого массива $R[j]$, он образует инверсию со всеми оставшимися элементами левого массива: $\text{inv} += (\text{len}_L - i)$.

18. Каково среднее и худшее время QuickSort?

Ответ: Среднее — $O(n \log n)$, худшее — $O(n^2)$ (при постоянном выборе экстремального pivot).

19. Как гарантировать расход памяти QuickSort не более $O(\log n)$?

Ответ: Делать рекурсивный вызов строго для меньшего подмассива, а больший обрабатывать циклом (устранение хвостовой рекурсии).

20. В чём разница между разбиением Хоара и Ломуто?

Ответ: Ломуто использует один бегущий указатель и делает много свопов. Хоар двигает два указателя навстречу друг другу, делает в ~ 3 раза меньше свопов и делит массив из одинаковых элементов пополам.

21. Какова сложность Counting Sort по времени и памяти?

Ответ: Время: $O(n + K)$, Память: $O(n + K)$, где K — диапазон ключей ($\max - \min + 1$).

22. Почему в устойчивом Counting Sort массив обходят справа налево?

Ответ: Чтобы элемент, стоявший правее в исходном массиве, занял наибольшую свободную позицию из отведенных для данного ключа, сохранив порядок дубликатов.

23. В чём разница между LSD и MSD поразрядной сортировкой?

Ответ: LSD сортирует от младших разрядов к старшим на всем массиве. MSD сортирует от старших к младшим, рекурсивно разбивая данные на корзины.

24. Почему для LSD Radix Sort критична строгая устойчивость подосновы?

Ответ: Если промежуточная сортировка неустойчива, сортировка по старшим разрядам разрушит порядок, уже сформированный по младшим разрядам.

25. Какова амортизированная стоимость push_back в динамическом массиве при факторе 2?

Ответ: $O(1)$ амортизированно, так как при расширении с N до $2N$ копирование N элементов распределяется на N предыдущих операций (стоимость 3 монеты на вставку).

26. Чем стек на односвязном списке уступает вектору на современном процессоре?

Ответ: Узлы списка разбросаны в куче (heap) $\rightarrow$ регулярные промахи кэша (cache misses). Оверхед на 64-битную ссылку и метаданные аллокатора в разы превышает размер данных.

27. Как кольцевой буфер очереди различает состояния «пуст» и «полон»?

Ответ: Либо хранением счетчика текущего размера `sz`, либо оставлением одной ячейки массива всегда пустой: пустота — `head == tail`, полнота — `(tail + 1) % N == head`.

28. Какова сложность удаления узла из односвязного списка по указателю на него?

Ответ: $O(n)$, если неизвестен предшественник (нужно найти его обходом от головы), либо $O(1)$ трюком с копированием данных из следующего узла (если узел не последний).

29. Какова сложность удаления узла из двусвязного списка по указателю?

Ответ: Строго $O(1)$ в худшем случае, так как узел хранит прямой указатель на своего предшественника `prev`.

30. Зачем в циклическом списке используют фиктивный сторожевой узел (Dummy/Sentinel)?

Ответ: Он исключает проверки на `nullptr` и граничные условия для пустых списков, делая код операций вставки и удаления единообразным.

31. Назовите формулы детей и родителя в куче при 0-индексации.

Ответ: Левый ребенок: $2i + 1$. Правый ребенок: $2i + 2$. Родитель: $\left\lfloor \frac{i-1}{2} \right\rfloor$.

32. Назовите формулы детей и родителя в куче при 1-индексации.

Ответ: Левый ребенок: $2i$. Правый ребенок: $2i + 1$. Родитель: $\left\lfloor \frac{i}{2} \right\rfloor$.

33. Какова сложность операций `siftUp` и `siftDown` в куче размера n?

Ответ: $O(\log n)$ — пропорциональна высоте завершенного бинарного дерева $h = \lfloor \log_2 n \rfloor$.

34. За какое время строится куча методом siftDown снизу вверх?

Ответ: За строго линейное время $O(n)$ благодаря сходящемуся ряду $\sum \frac{h}{2^h} = 2$.

35. Почему построение кучи через n вызовов siftUp сверху вниз занимает $O(n \log n)$?

Ответ: Бóльшая часть вершин ($\sim \frac{n}{2}$) — это листья на максимальной глубине $\log n$, и каждая из них поднимается на высоту $\log n$.

36. Устойчива ли HeapSort? In-place ли она?

Ответ: HeapSort **неустойчива** (дальние перестановки в куче сбивают порядок равных ключей), но строго **In-place** ($O(1)$ доп. памяти).

37. Как удалить произвольный элемент из кучи по индексу i за $O(\log n)$?

Ответ: Заменить $A[i] = A[n-1]$, уменьшить размер кучи на 1, затем вызвать `siftUp(i)` или `siftDown(i)` в зависимости от сравнения с предком.

38. Как реализовать decreaseKey в куче за $O(\log n)$?

Ответ: Уменьшить значение в ячейке, вызвать `siftDown` (для Max-Heap) или `siftUp` (для Min-Heap), синхронно обновляя позиции в обратном массиве `pos[id]`.

39. Каковы два признака применимости динамического программирования?

Ответ:
1. Оптимальная подструктура (оптимальное решение задачи выражается через оптимальные решения подзадач). 2. Перекрывающиеся подзадачи.

40. Какова сложность задачи о Кузнечике с прыжком до k кочек (наивно и оптимально)?

Ответ: Наивно: $O(nk)$ времени. Оптимально: $O(n)$ времени с поддержкой скользящего максимума через монотонный дек.

41. Каково состояние и рекуррента в 0-1 рюкзаке?

Ответ: $\mathrm{dp}[i][w] = \max(\mathrm{dp}[i-1][w], \mathrm{dp}[i-1][w-w_i] + c_i)$. Время $O(nW)$, память $O(nW)$.

42. Почему в 0-1 рюкзаке при оптимизации памяти до $O(W)$ обход идет справа налево?

Ответ: Чтобы состояние $\mathrm{dp}[w-w_i]$ бралось с **предыдущей** итерации (пока предмет i ещё не учтен), исключая повторное взятие одного предмета.

43. Какова асимптотика НВП (LIS) с бинарным поиском?

Ответ: $O(n \log n)$ времени и $O(n)$ памяти.

44. Почему массив хвостов d в алгоритме НВП строго монотонно возрастает?

Ответ: Минимальный конец возрастающей подпоследовательности длины L обязан быть строго больше минимального конца для длины $L-1$, иначе подпоследовательность длины $L-1$ можно было бы закончить ещё меньшим числом.

45. Каково худшее время работы алгоритма BFPRT (медиана медиан)?

Ответ: Гарантированное линейное время $O(n)$ в худшем случае за счет выбора групп по 5 элементов.

46. Назовите 3 случая мастер-теоремы для $T(n) = aT\left(\frac{n}{b}\right) + f(n)$.

Ответ:
1. $f(n) = O(n^c), c < \log_b a \to \Theta\left(n^{\log_b a}\right)$.
2. $f(n) = \Theta\left(n^{\log_b a} \log^k n\right) \to \Theta\left(n^{\log_b a} \log^{k+1} n\right)$.
3. $f(n) = \Omega(n^c), c > \log_b a \to \Theta(f(n))$.

47. Как работает очередь с минимумом на двух стеках?

Ответ: Один стек на вход (S_{in}), второй на выход (S_{out}). Каждый стек поддерживает свой минимум. При опустошении S_{out} элементы переливаются из S_{in} с разворотом.

48. Какова амортизированная сложность getMin в очереди на двух стеках?

Ответ: Строго $O(1)$ в худшем случае, так как это просто $\min(S_{\text{in}}.\text{getMin}(), S_{\text{out}}.\text{getMin}())$.

49. Почему в бинарном поиске нельзя писать `m = (L + R) / 2`?

Ответ: При больших индексах (например, $L, R \approx 1.5 \cdot 10^9$) сумма $L + R$ вызовет 32-битное знаковое переполнение `int` в отрицательные числа. Нужно: `L + (R - L) / 2`.

50. Почему в вещественном бинарном поиске надежнее делать 100 итераций, чем проверять `R - L > EPS`?

Ответ: При плохом выборе EPS или больших числах условие `R - L > EPS` может зациклиться из-за потери точности мантиссы IEEE 754. 100 итераций делят отрезок в $2^{100} \approx 10^{30}$ раз без риска зацикливания.

ЧАСТЬ 2. ГЛУБОКИЕ ТЕОРЕТИЧЕСКИЕ БИЛЕТЫ (РАЗВЁРНУТЫЕ ЭТАЛОННЫЕ ОТВЕТЫ)

Билет №1: Асимптотический анализ сложности по времени и модели вычислений

Темы 1, 2

1. Понятие алгоритма и модели вычислений: Алгоритм анализируется в рамках формальной модели вычислений. В курсе используется модель **word RAM** (модель машины с произвольным доступом к памяти и машинными словами длины $w \geq \log_2 n$ бит). В этой модели:
- Чтение и запись в любую ячейку памяти по адресу выполняется за $O(1)$ такт.
- Арифметические (+, -, *, /) и побитовые (&, |, ^, <<, >>) операции над словами выполняются за $O(1)$.

2. Функция трудоёмкости $T(n)$: число атомарных шагов машины на худшем входе размера n. **3. Формальные определения асимптотических классов:**
- $O(g(n)) = \{f(n) : \exists c > 0, n_0 : \forall n \geq n_0, 0 \leq f(n) \leq cg(n)\}$
- $\Omega(g(n)) = \{f(n) : \exists c > 0, n_0 : \forall n \geq n_0, f(n) \geq cg(n) \geq 0\}$
- $\Theta(g(n)) = O(g(n)) \cap \Omega(g(n))$
- $o(g(n)) = \{f(n) : \forall c > 0, \exists n_0 : \forall n \geq n_0, 0 \leq f(n) < cg(n)\} \Leftrightarrow \lim_{n\to\infty} \frac{f(n)}{g(n)} = 0$

4. Предельный критерий: если $\lim_{n\to\infty} \frac{f(n)}{g(n)} = L$:
- $L = 0 \to f \in o(g) \subset O(g)$;
- $0 < L < \infty \to f \in \Theta(g)$;
- $L = \infty \to f \in \omega(g) \subset \Omega(g)$.

Билет №2: Анализ сложности алгоритмов по памяти

Тема 2

1. Структура памяти процесса: Вход/выход (input/output) и дополнительная рабочая память (auxiliary space). Оценка алгоритма даётся именно по auxiliary space. **2. Стек вызовов (Call Stack):** при каждом вызове функции создаётся стековый кадр фиксированного размера c. **3. Связь глубины рекурсии и памяти:** если максимальная глубина дерева рекурсии равна $h(n)$, то суммарный расход стековой памяти составляет $\Theta(h(n))$.
- Пример: для сбалансированного деления пополам (MergeSort) глубина $h = \log_2 n \to O(\log n)$ стека.
- Пример: наивный QuickSort на отсортированном массиве порождает цепочку глубины $n \to O(n)$ стека (риск Stack Overflow).

4. Устранение хвостовой рекурсии (Tail Call Optimization): рекурсивный вызов делается только для меньшей подзадачи ($\leq \frac{n}{2}$), а большая обрабатывается в цикле $\to$ глубина стека гарантированно $\leq \log_2 n$.

Билет №3: Сортировка вставками: анализ числа инверсий и адаптивность

Тема 3

1. Идея и инвариант: Подмассив $A[0..i-1]$ всегда отсортирован. На шаге i элемент $x = A[i]$ сдвигает вправо все элементы $A[j] > x$ и встаёт на своё место. **2. Связь с инверсиями:** Инверсией называется пара (i, j), где $i < j$, но $A[i] > A[j]$.
- Каждый сдвиг элемента $A[j]$ вправо в процессе вставки x устраняет ровно одну инверсию между x и $A[j]$.
- Число сравнений: $C(n) = I + (n-1) - E$, где I — число инверсий, E — число случаев остановки в начале массива.
- Итоговое время: $\Theta(n + I)$.

3. Анализ случаев:
- Лучший случай: массив отсортирован, $I = 0 \to \Theta(n)$.
- Худший случай: массив обратно отсортирован, $I = \frac{n(n-1)}{2} \in \Theta(n^2)$.
- Средний случай: матожидание числа инверсий случайной перестановки $E[I] = \frac{n(n-1)}{4} \in \Theta(n^2)$.

4. Свойства: Устойчива, in-place, адаптивна. Идеальна для малых массивов ($n \leq 16$).

Билет №4: Сортировка слиянием: рекурсивный и итеративный варианты, инверсии

Тема 4

1. Принцип Divide and Conquer: Рекурсивно делим массив пополам до $n = 1$, затем сливаем упорядоченные части за $O(n)$. **2. Рекуррента времени:** $T(n) = 2T(\frac{n}{2}) + \Theta(n)$. По мастер-теореме (случай 2, $k = 0$): $T(n) \in \Theta(n \log n)$ всегда. **3. Память:** $O(n)$ под вспомогательный массив для слияния. **4. Итеративный MergeSort (снизу вверх):** без рекурсии объединяет смежные блоки размера $1, 2, 4, 8...$ с помощью циклов. **5. Подсчёт инверсий за $O(n \log n)$:** при слиянии подмассивов L и R, если элемент берется из $R[j]$, то он меньше всех оставшихся элементов $L[i..|L| - 1]$. Добавляем к общему счетчику $(|L| - i)$ инверсий.

Билет №5: Быстрая сортировка: схемы Хоара и Ломуто, худший случай и память

Тема 5

1. Общая схема: Выбираем pivot, разбиваем массив на две части ($\leq$ pivot и $\geq$ pivot), рекурсивно сортируем. **2. Сравнение схем разбиения:**

- **Lomuto:** один указатель j сканирует массив, элементы $<$ pivot переставляются в левую часть. Делает много свопов. На массиве из одинаковых элементов делит массив в соотношении $0 : n - 1$, приводя к $O(n^2)$!
- **Hoare:** указатели i и j движутся навстречу друг другу. Делает в среднем в 3 раза меньше свопов. На массиве из одинаковых элементов делит массив строго пополам!

3. Худший случай: $T(n) = T(n - 1) + \Theta(n) = \Theta(n^2)$ при постоянном выборе минимального/максимального элемента. **4. Защита:** Рандомизированный выбор pivot или медиана трех (first, middle, last). Ограничение глубины стека до $O(\log n)$ через снятие хвостовой рекурсии для большего куска.

Билет №6: Сортировка подсчетом: алгоритм с префиксными суммами и устойчивость

Тема 6

1. Постановка: Массив $A[0..n - 1]$ целых чисел из отрезка $[0, K - 1]$. **2. Шаги устойчивой версии:**

1. Массив частот $C[0..K - 1] = 0$. Подсчет: для каждого $x \in A$ делаем $C[x] + +$ (время $O(n)$).
2. Префиксные суммы: $C[i] = C[i] + C[i - 1]$ для $i = 1...K - 1$ (время $O(K)$). $C[v]$ хранит число элементов $\leq v$.
3. Раскладка в выходной массив B: цикл по i от $n - 1$ вниз до 0: $B[C[A[i]] - 1] = A[i]; C[A[i]] - -$.

3. Доказательство устойчивости: Элементы с равными ключами берутся справа налево и размещаются в свободные ячейки также справа налево (с максимального индекса до минимального), сохраняя исходный относительный порядок. **4. Сложность:** Время $\Theta(n + K)$, Память $\Theta(n + K)$. Эффективна при $K = O(n)$.

Билет №7: Цифровая (поразрядная) сортировка: LSD vs MSD

Тема 7

1. Идея Radix Sort: Ключи представляются как строки из d разрядов в системе счисления по основанию B. **2. LSD (Least Significant Digit):**

- Сортирует последовательно по разрядам от младшего (0) к старшему ($d - 1$).
- На каждом шаге используется **строго устойчивая** сортировка (обычно Counting Sort по текущему разряду).
- **Инвариант:** после фазы k элементы отсортированы по младшим $k + 1$ разрядам.
- Сложность: Время $O(d(n + B))$, Память $O(n + B)$. Для 32-битных int при $B = 256$, $d = 4 \rightarrow$ линейное время!

3. MSD (Most Significant Digit):

- Сортирует от старших разрядов к младшим. Рекурсивно разбивает массив на B корзин (buckets) и запускается в каждой.
- Требует управления памятью и рекурсией, не требует устойчивости внутри одной корзины.

Билет №8: Абстрактный тип данных Стек: реализации и амортизация

Тема 8, 13

1. Спецификация АТД Стек: Принцип LIFO. Операции `push(x)`, `pop()`, `top()`, `isEmpty()`. **2. Реализация на динамическом массиве:**

- При заполнении массива емкости N выделяется буфер размера $2N$, элементы копируются, старый буфер освобождается.

- **Амортизационный анализ методом монет (бухгалтерский метод):**
  - Реальная стоимость обычной вставки без расширения: 1 операция.
  - Назначаем учетную стоимость 3 монеты: 1 монета платит за текущую запись, 1 резервируется для будущего копирования этого элемента, 1 — для копирования одного из ранее вставленных элементов.
  - К моменту заполнения массива размера $2N$ накоплено $2N$ монет, чего точно хватает на копирование всех элементов.
  - Итоговая амортизированная стоимость: $O(1)$.

3. Реализация на односвязном списке:
- Вставка и удаление в голове списка. Гарантированное худшее время $O(1)$ для каждого шага.
- Минусы: высокий оверхед по памяти (8 байт ссылка $+16$ байт аллокатор на элемент) и промахи кэша CPU.

Билет №9: Очередь: кольцевой буфер и связный список

Тема 9, 14

1. Спецификация АТД Очередь: Принцип FIFO. Операции `enqueue(x)`, `dequeue()`, `front()`, `isEmpty()`. **2. Кольцевой буфер (Ring Buffer):**
- Непрерывный массив размера N, индексы `head` и `tail`.
- Продвижение индексов по модулю: `tail = (tail + 1) % N`, `head = (head + 1) % N`.
- Условие полноты буфера: при наличии поля `size` — `size == N`. Без поля `size` — `(tail + 1) % N == head` (одна ячейка резервируется).
- При переполнении: выделяется массив размера $2N$, элементы переносятся в порядке от `head` до `tail` в индексы $0...N-1$.

3. Реализация на связном списке: Два указателя: `head` (откуда берем) и `tail` (куда кладем). Добавление и удаление за строгое $O(1)$ без единовременных пауз на реаллокацию.

Билет №10: Односвязный список: асимптотики операций и кэш-эффекты

Тема 10

1. Структура узла: `struct Node { T val; Node* next; };`. **2. Асимптотика операций:**
- `push_front` / `pop_front`: $O(1)$ всегда.
- `insert_after` / `delete_after`: $O(1)$ при наличии указателя на узел.
- Поиск по значению или доступ по индексу `[i]`: $\Theta(n)$ последовательным проходом.
- `pop_back`: $\Theta(n)$, так как невозможно обновить `next` предпоследнего узла за $O(1)$.

3. Проблема локальности кэша:
- Массивы располагаются в памяти непрерывно: процессор считывает целую кэш-линию (64 байта), обеспечивая $O(1)$ доступ к соседним элементам с аппаратным предвыбором (hardware prefetcher).
- Узлы списка аллоцируются в произвольных местах виртуальной памяти, что вызывает задержку в сотни тактов на каждое разыменование указателя (`cache miss`).

Билет №11: Двусвязный список: удаление за O(1) и реализация LRU-кэша

Тема 11

1. Структура узла: `struct Node { T val; Node* prev; Node* next; };`. **2. Удаление произвольного узла P за $O(1)$:** `P->prev->next = P->next; P->next->prev = P->prev; delete P;`. Знание предшественника не требует поиска! **3. Архитектура LRU-кэша (Least Recently Used Cache):**
- Задача: хранить C последних запрошенных элементов. При переполнении удалять элемент, к которому обращались дольше всего.
- Решение: Двусвязный список + Хеш-таблица (`std::unordered_map<Key, Node*>`).
- При обращении к ключу `get(key)`: находим узел в хеш-таблице за $O(1)$, вырезаем из текущей позиции списка за $O(1)$ и переставляем в голову списка (`push_front`) за $O(1)$.
- При добавлении `put(key, val)`: если размер $> C$, удаляем хвостовой узел списка за $O(1)$ и стираем его из хеш-таблицы.

Билет №12: Циклический список и техника фиктивного узла (Sentinel)

Тема 12

1. Топология: Последний элемент указывает на первый (`tail->next = head`). **2. Сторожевой фиктивный узел (Sentinel / Dummy):**
- В кольцо помещается постоянный узел, не содержащий полезных данных.
- Список никогда не бывает пустым: пустой список — это сторож, указывающий сам на себя (`sentinel->next = sentinel`).
- Исчезают любые проверки `if (head == nullptr)` и обновления указателя `head` при вставках/удалениях.

3. Задача Иосифа Флавия (n человек в кругу, выбывает каждый k-й):
- Моделируется циклическим списком за $O(nk)$ шагов с удалением узлов за $O(1)$.

Билет №13: Сравнение стека на списках и динамическом массиве

Тема 13

1. Временные гарантии:
- Связный список: строго $O(1)$ в худшем случае для каждого вызова. Нет лагов на перевыделение памяти.
- Динамический массив: амортизированное $O(1)$. Редкие операции удвоения требуют $O(n)$ времени.

2. Анализ производительности и памяти:
- Для хранения 1 миллиона 4-байтных `int`:
  - В векторе: 4 МБ данных $+ \sim 4$ МБ резерва емкости $= 8$ МБ непрерывной памяти.
  - В списке: $10^6 \times (4 + 8\ (\text{next}) + 16\ (\text{malloc header})) = 28$ МБ фрагментированной памяти!
- Динамический массив на порядки быстрее на практике за счет эффективного использования кэша L1/L2.

Билет №14: Сравнение очереди на связном списке и циклическом массиве

Тема 14

1. Связный список: Два указателя `head` и `tail`. Нет ограничения по максимальной емкости. Каждая вставка — аллокация в куче. **2. Кольцевой буфер:** Память выделяется крупными блоками. Минимальные накладные расходы, превосходная скорость. **3. Вывод:** Кольцевой буфер является стандартом де-факто для высокопроизводительных очередей и сетевых драйверов (Ring Buffer сетевой карты).

Билет №15: Двоичная куча: структура, индексация и восстановление свойств

Тема 15

1. Логическая структура: Почти полное (завершенное) бинарное дерево высоты $\lfloor \log_2 n \rfloor$. **2. Физическое представление в массиве:**
- Узел $i \to$ Левый сын $2i + 1$, Правый сын $2i + 2$, Отец $\left\lfloor \frac{i-1}{2} \right\rfloor$.

3. Инвариант Max-Heap: $A[\text{parent}(i)] \geq A[i]$ для всех $i > 0$. **4. Просеивание вверх (siftUp):** элемент всплывает вверх, обмениваясь с родителем, пока нарушается инвариант. Высота дерева $\log_2 n \to O(\log n)$. **5. Просеивание вниз (siftDown):** элемент опускается вниз, обмениваясь с **наибольшим из двух детей**, пока меньше хотя бы одного из них. Время $O(\log n)$.

Билет №16: Теорема о построении кучи за O(n) (Build Heap)

Тема 16

1. Алгоритм: Вызов `siftDown(i)` для всех нелистовых вершин от $\lfloor \frac{n}{2} \rfloor - 1$ вниз до 0. **2. Строгое доказательство:**
- Высота узла h — расстояние до листьев.
- Количество узлов на высоте h не превышает $\left\lceil \frac{n}{2^{h+1}} \right\rceil$.
- Стоимость `siftDown` на высоте h равна $O(h)$.
- Суммарное время:

$$T(n) \leq \sum_{h=0}^{\lfloor \log_2 n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil \cdot ch \leq cn \sum_{h=1}^{\infty} \frac{h}{2^{h+1}} = \frac{cn}{2} \sum_{h=1}^{\infty} \frac{h}{2^h}$$

- Вычисление суммы ряда $S = \sum_{h=1}^{\infty} \frac{h}{2^h}$:

$$S = \frac{1}{2} + \frac{2}{4} + \frac{3}{8} + \frac{4}{16} + ...$$

$$\frac{1}{2}S = \frac{1}{4} + \frac{2}{8} + \frac{3}{16} + ...$$

$$S - \frac{1}{2}S = \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \frac{1}{16} + ... = 1 \to \frac{1}{2}S = 1 \to S = 2.$$

- Подставляя $S = 2$: $T(n) \leq \frac{cn}{2} \cdot 2 = cn \in O(n)$.

Билет №17: Пирамидальная сортировка (HeapSort)

Тема 17

1. Шаги:

1. Построение Max-Heap из массива за $O(n)$ методом `siftDown`.
2. Цикл по i от $n-1$ вниз до 1:
   - `swap(A[0], A[i])` — максимум переносится в конец;
   - размер кучи уменьшается до i;
   - `siftDown(0)` на префиксе длины i за $O(\log i)$.

2. Сложность: Время $O(n) + \sum_{i=1}^{n} O(\log i) = \Theta(n \log n)$ всегда. Память $O(1)$ in-place. Неустойчива. **3. Сравнение с QuickSort:** HeapSort не имеет худшего случая $O(n^2)$, но на практике медленнее в 2–3 раза из-за непоследовательного доступа к памяти (прыжки по индексам $2i + 1$).

Билет №18: Удаление элементов из двоичной кучи

Тема 18

1. Извлечение максимума (extractMax):

- Корень $A[0]$ заменяется последним элементом $A[n-1]$.
- Размер уменьшается на 1.
- Вызывается `siftDown(0)`. Время $O(\log n)$.

2. Удаление произвольного элемента по индексу i:

- Заменяем $A[i] = A[n-1]$, уменьшаем размер кучи на 1.
- Сравниваем новое значение $A[i]$ с родителем:
  - Если $A[i] > A[\text{parent}(i)]$, вызываем `siftUp(i)`.
  - Иначе вызываем `siftDown(i)`.
- Время $O(\log n)$.

Билет №19: Очередь с приоритетом и изменение приоритета ключа

Тема 19

1. Операции: `insert(x, p)`, `extractMax()`, `increaseKey(id, new_p)`, `decreaseKey(id, new_p)`. **2. Реализация изменения ключа:**

- Чтобы найти элемент в куче по его идентификатору id за $O(1)$, заводится внешний массив или хеш-таблица `pos[id] = index`.
- При любых обменах элементов в куче `swap(A[i], A[j])` обновляются индексы: `swap(pos[A[i].id], pos[A[j].id])`.
- Изменение ключа вызывает `siftUp` или `siftDown` за $O(\log n)$.

Билет №20: Динамическое программирование: основы и классические задачи

Тема 20

1. Принцип оптимальности Беллмана: Оптимальная траектория состоит из оптимальных подтраекторий. **2. Кузнечик:** $\mathrm{dp}[i] = c_i + \max_{1 \le j \le k} \mathrm{dp}[i-j]$. Время $O(nk)$ наивно, $O(n)$ с очередью с максимумом. **3. Рюкзак 0-1:** $\mathrm{dp}[i][w] = \max(\mathrm{dp}[i-1][w], \mathrm{dp}[i-1][w-w_i] + c_i)$.

- 1D оптимизация: $\mathrm{dp}[w]$ обходится справа налево (от W до w_i), предотвращая многократный учет предмета.

4. НВП (LIS):

- За $O(n^2)$: $\mathrm{dp}[i] = 1 + \max_{j<i, A[j]<A[i]} \mathrm{dp}[j]$.
- За $O(n \log n)$: массив $d[\mathrm{len}]$ — минимальный конец возрастающей подпоследовательности длины len. Строгая монотонность d позволяет искать позицию через `lower_bound` за $O(\log n)$.

Билет №*1: Квикселект: детерминированный алгоритм медианы медиан (BFPRT)

Тема *1

1. Постановка: Поиск k-й порядковой статистики за $O(n)$ в худшем случае. **2. Шаги BFPRT:**

1. Разбить массив на блоки по 5 элементов.
2. Найти медиану каждого блока (сортировкой за $O(1)$).
3. Рекурсивно найти медиану медиан M.
4. Разбить массив вокруг M (partition).
5. Рекурсивно перейти в одну из половин.

3. Доказательство: Число элементов $\le M$ не менее $3 \cdot \left(\frac{1}{2} \cdot \frac{n}{5} - 2\right) \ge \frac{3n}{10} - 6$. Размер подзадачи $\le \frac{7n}{10} + 6$. $T(n) \le T\left(\frac{n}{5}\right) + T\left(7\frac{n}{10}\right) + O(n)$. Так как $\frac{1}{5} + \frac{7}{10} = \frac{9}{10} < 1$, то $T(n) \in O(n)$.

Билет №*2: Мастер-теорема: формулировка, дерево рекурсии и границы применимости

Тема *2

1. Формула: $T(n) = aT\left(\frac{n}{b}\right) + f(n)$. Критическое значение $c_{\text{crit}} = \log_b a$. **2. Случаи:**

1. $f(n) = O(n^c), c < c_{\text{crit}} \to \Theta\left(n^{\log_b a}\right)$ (листья доминируют).
2. $f(n) = \Theta\left(n^{c_{\text{crit}}} \log^k n\right) \to \Theta\left(n^{c_{\text{crit}}} \log^{k+1} n\right)$ (равенство уровней).
3. $f(n) = \Omega(n^c), c > c_{\text{crit}}$ и $af\left(\frac{n}{b}\right) \le c' f(n) \to \Theta(f(n))$ (корень доминирует).

3. Ограничения: не применима при непостоянных a, b или неполиномиальном разрыве между $f(n)$ и $n^{c_{\text{crit}}}$.

Билет №*3: Дин прог: битовые маски, профиль и редакционное расстояние

Тема *3

1. Битовые маски: хранение подмножеств в целых числах. Задача TSP за $O(n^2 2^n)$. **2. Динамика по профилю:** замощение сетки $N \times M$ домино. Состояние $\mathrm{dp}[i][\mathrm{mask}]$. Поклеточная динамика (изломанный профиль) работает за $O(NM2^N)$. **3. Редакционное расстояние (Левенштейн):** $\mathrm{dp}[i][j] = \min(\mathrm{dp}[i-1][j] + 1, \mathrm{dp}[i][j-1] + 1, \mathrm{dp}[i-1][j-1] + [S_1[i-1] \ne S_2[j-1]])$. Время $O(nm)$, память $O(\min(n, m))$.

Билет №*4: Дек и Очередь с поддержкой минимума за O(1)

Тема *4

1. Стек с минимумом: хранит пары `(val, min_so_far)`. Все операции за строгое $O(1)$. **2. Очередь с минимумом на двух стеках:**

- S_{in} принимает новые элементы.
- S_{out} отдает элементы. Если S_{out} пуст, все элементы переливаются из S_{in} в S_{out} с разворотом порядка.
- Минимум очереди: $\min(S_{\text{in}}.\mathrm{getMin}(), S_{\text{out}}.\mathrm{getMin}())$ за $O(1)$.
- Амортизация: каждый элемент участвует ровно в 1 переносе $\to O(1)$ амортизированно.

3. Скользящее окно минимума: за $O(n)$ находит минимумы во всех окнах длины k.

Билет №*5: Бинарный поиск: lower_bound, по ответу и вещественный

Тема *5

1. Инвариант полуинтервала $[L, R)$: середина $m = L + \frac{R-L}{2}$. Ответ всегда лежит внутри диапазона. `lower_bound` возвращает наименьший индекс i, где $A[i] \geq x$. **2. Бинпоиск по ответу:** применим при монотонности предиката $\mathrm{check}(X)$. Время $O(\log(\mathrm{Range}) \cdot T_{\mathrm{check}})$. **3. Вещественный бинпоиск:** поиск корня $f(x) = 0$. Рекомендуется ровно 100 итераций деления отрезка пополам для абсолютной численной устойчивости.

ЧАСТЬ 3. ЛОВУШКИ ЭКЗАМЕНАТОРОВ И КАВЕРЗНЫЕ ВОПРОСЫ (25 ЛОВУШЕК)

⚠️ Ловушка №1: Верно ли, что $2^{n+1} \in O(2^n)$? А $2^{2n} \in O(2^n)$?

В чём подвох (где ошибаются): Студенты механически считают, что любая константа в степени «отбрасывается».

Эталонный ответ:
1. $2^{n+1} = 2 \cdot 2^n$. Здесь $c = 2$, условие $2^{n+1} \le c \cdot 2^n$ выполнено при $c = 2, n_0 = 1$. Ответ: **ВЕРНО**.
2. $2^{2n} = (2^2)^n = 4^n$. Отношение $\frac{4^n}{2^n} = 2^n \to \infty$. Константы c не существует. Ответ: **НЕВЕРНО**, $2^{2n} \in \omega(2^n)$.

⚠️ Ловушка №2: Может ли алгоритм иметь худшее время $O(n)$, а лучший случай $\Omega(n^2)$?

В чём подвох (где ошибаются): Путаница понятий лучшего/худшего случая с O и Ω нотациями.

Эталонный ответ: НЕТ, это математически невозможно. Худший случай — это максимум времени работы по всем входам размера n, а лучший — минимум. Всегда $T_{\text{best}(n)} \le T_{\text{worst}(n)}$. Если бы $T_{\text{best}} \in \Omega(n^2)$, то $T_{\text{worst}} \ge T_{\text{best}} \ge cn^2$, что противоречит $T_{\text{worst}} \in O(n)$.

⚠️ Ловушка №3: Что будет, если в Counting Sort массив обойти слева направо, а счетчик декрементировать?

В чём подвох (где ошибаются): Проверка понимания механизма устойчивости.

Эталонный ответ: Алгоритм отсортирует массив правильно, но **потеряет устойчивость** (инвертирует порядок одинаковых элементов на противоположный). Первым записанный элемент получит самый большой доступный индекс, а последний — наименьший.

⚠️ Ловушка №4: Почему Lomuto Partition деградирует до $O(n^2)$ на массиве из всех одинаковых элементов, а Hoare — нет?

В чём подвох (где ошибаются): Незнание нюансов обработки элементов, равных pivot.

Эталонный ответ: В схеме Lomuto условие $A[j] \le \text{pivot}$ истинно для всех элементов, поэтому все n элементов попадают в одну половину, а вторая остается пустой (разбиение $n-1:0$). В схеме Hoare оба указателя останавливаются на элементах, равных pivot, и делают своп, заставляя указатели встретиться строго посередине (разбиение $\frac{n}{2} : \frac{n}{2}$).

⚠️ Ловушка №5: Почему наивный QuickSort падает со Stack Overflow на отсортированном массиве длины 10^5?

В чём подвох (где ошибаются): Игнорирование глубины стека вызовов.

Эталонный ответ: При выборе первого элемента в качестве pivot на отсортированном массиве на каждом шаге отсекается 1 элемент. Глубина рекурсии равна 10^5. Размер стека процессов обычно 8 МБ. 10^5 стековых кадров по $64 - 128$ байт переполняют стек потока.

⚠️ Ловушка №6: Можно ли отсортировать связный список быстрой сортировкой за $O(n \log n)$?

В чём подвох (где ошибаются): Применение массивно-ориентированных алгоритмов к спискам.

Эталонный ответ: Теоретически можно (схему Ломуто легко реализовать на односвязном списке), но практически крайне неэффективно: нет произвольного доступа для выбора хорошего pivot, частые промахи кэша. Для списков стандартом является MergeSort.

⚠️ Ловушка №7: В доказательстве Build Heap за $O(n)$: почему нельзя просто сказать, что в куче n элементов, и каждый просеивается за $O(\log n)$?

В чём подвох (где ошибаются): Разница между тривиальной верхней границей и точной асимптотикой.

Эталонный ответ: Оценка $n \cdot O(\log n) = O(n \log n)$ **верна, но не точна** (она завышена). В методе `siftDown` большинство вершин находятся на малых высотах: $\frac{n}{2}$ листьев имеют высоту 0 и не просеиваются вовсе, $\frac{n}{4}$ вершин имеют высоту 1 и делают 1 шаг. Точный суммарный ряд сходится к $O(n)$.

⚠️ Ловушка №8: Что произойдет, если в 0-1 рюкзаке с одномерным массивом идти по весу слева направо ($w = w_i ... W$)?

В чём подвох (где ошибаются): Понимание различия между 0-1 рюкзаком и неограниченным рюкзаком.

Эталонный ответ: Мы решим **задачу о неограниченном рюкзаке (Unbounded Knapsack)**, где каждый предмет можно брать бесконечное число раз! Обновленное значение $\mathrm{dp}[w - w_i]$ с текущего шага будет повторно использовано при подсчете $\mathrm{dp}[w]$, добавив предмет i ещё раз.

⚠️ Ловушка №9: Почему в LIS $O(n \log n)$ массив хвостов d НЕ хранит саму наибольшую подпоследовательность?

В чём подвох (где ошибаются): Студенты думают, что массив d в конце работы — это и есть ответ.

Эталонный ответ: Массив d хранит **минимальные возможные концы** подпоследовательностей разных длин на текущем префиксе. Элементы в d могут браться из разных несовместимых подпоследовательностей! Для восстановления ответа необходимо хранить массивы предков `prev[i]` и индексов `pos[len]`.

⚠️ Ловушка №10: Почему мастер-теорема не применима к $T(n) = 2T(\frac{n}{2}) + n \log n$ в базовой формулировке Кормена?

В чём подвох (где ошибаются): Тонкость условия полиномиального различия.

Эталонный ответ: Здесь $\log_b a = \log_2 2 = 1$, а $f(n) = n \log n$. Функция $f(n)$ асимптотически больше, чем n^1, но различие выражается логарифмом, а не полиномиальным множителем n^ε. Базовая мастер-теорема неприменима. Однако применима расширенная мастер-теорема (случай 2 с $k = 1$), дающая $\Theta(n \log^2 n)$.

⚠️ Ловушка №11: Почему в детерминированном Quickselect деление на тройки не дает $O(n)$?

В чём подвох (где ошибаются): Математический расчет сходимости рекурренты.

Эталонный ответ: При делении на тройки медиана гарантирует отсечение не менее $2 \cdot (\frac{n}{6}) = \frac{n}{3}$ элементов. Размер оставшейся части $\leq 2\frac{n}{3}$. Рекуррента: $T(n) \leq T(\frac{n}{3}) + T(2\frac{n}{3}) + O(n)$. Так как коэффициенты $\frac{1}{3} + \frac{2}{3} = 1$, то сумма по уровням дерева рекурсии одинакова на всех уровнях, что дает $T(n) \in \Theta(n \log n)$, а не $O(n)$!

⚠️ Ловушка №12: Почему в бинарном поиске `m = (L + R) / 2` приводит к UB или вылету?

В чём подвох (где ошибаются): Проблема 32-битного целочисленного переполнения.

Эталонный ответ: Если $L + R > 2^{31} - 1 \approx 2.14 \cdot 10^9$, сложение дает переполнение со знаковым минусом (`negative index`), что вызывает `Segmentation fault` при обращении `A[m]`. Правильно: `m = L + (R - L) / 2`.

⚠️ Ловушка №13: Почему амортизированное $O(1)$ вектора неприемлемо в системах реального времени (ABS, авиация)?

В чём подвох (где ошибаются): Разница между средней и худшей ценой операции.

Эталонный ответ: В жестком реальном времени (Hard Real-Time) существует строгий дедлайн (например, 1 мс на реакцию тормоза). Операция `push_back`, попавшая на момент удвоения массива с 10^6 до $2 \cdot 10^6$ элементов, вызовет паузу в десятки миллисекунд для копирования памяти, что приведет к катастрофе.

⚠️ Ловушка №14: Можно ли реализовать очередь с минимумом на одном стеке?

В чём подвох (где ошибаются): Понимание различия LIFO и FIFO.

Эталонный ответ: Нет, стек не может изменить порядок извлечения на противоположный. Для реализации FIFO из LIFO математически необходимы два стека (один инвертирует порядок при переливании).

⚠️ Ловушка №15: Какова сложность поиска элемента с заданным значением в двоичной куче?

В чём подвох (где ошибаются): Кучу путают с двоичным деревом поиска (BST).

Эталонный ответ: Сложность $\Theta(n)$! В куче нет порядка между левым и правым ребенком (мы знаем только, что оба меньше родителя). Поэтому двоичный поиск невозможен — требуется полный линейный обход всех узлов.

⚠️ Ловушка №16: Почему при удалении элемента из кучи мы вызываем либо siftUp, либо siftDown, но никогда оба?

В чём подвох (где ошибаются): Инварианты кучи.

Эталонный ответ: Мы заменили элемент $A[i]$ последним элементом кучи $A[n-1]$. Значение $A[n-1]$ может быть либо больше своего нового родителя (тогда нужно поднимать вверх через `siftUp`), либо меньше хотя бы одного из детей (тогда опускать вниз через `siftDown`), либо находиться в допустимых границах. Нарушение одновременно сверху и снизу невозможно, так как до удаления куча была корректной.

⚠️ Ловушка №17: Устойчива ли цифровая сортировка Radix LSD, если в качестве поразрядной сортировки взять HeapSort?

В чём подвох (где ошибаются): Критическая зависимость от устойчивости шага.

Эталонный ответ: КАТЕГОРИЧЕСКИ НЕТ. HeapSort неустойчива. При сортировке по следующему разряду порядок элементов с одинаковым значением разряда перемешается, что полностью разрушит сортировку по предыдущим (младшим) разрядам.

⚠️ Ловушка №18: В чём разница между `lower_bound` и `upper_bound` на массиве `[1, 2, 2, 2, 3]` для числа 2?

В чём подвох (где ошибаются): Понимание границ интервала дубликатов.

Эталонный ответ:
- `lower_bound` возвращает итератор на **первую двойку** (индекс 1) — первый элемент, который ≥ 2.
- `upper_bound` возвращает итератор на **тройку** (индекс 4) — первый элемент, который строго > 2.
- Полуинтервал `[lower_bound, upper_bound)` задает диапазон всех вхождений числа 2!

⚠️ Ловушка №19: Почему в вещественном бинпоиске условие `while (R - L > EPS)` опасно вечным циклом?

В чём подвох (где ошибаются): Ограничения стандарта чисел с плавающей точкой IEEE 754.

Эталонный ответ: При больших числах расстояние между соседними представимыми числами `double` (машинный эпсилон) может стать больше, чем заданный `EPS`. Тогда `(L + R) / 2.0` из-за округления совпадет с `L` или `R`, отрезок перестанет уменьшаться, и цикл никогда не завершится.

⚠️ Ловушка №20: Почему динамика по профилю неприменима для сетки 100×100?

В чём подвох (где ошибаются): Экспоненциальная зависимость от ширины профиля.

Эталонный ответ: Сложность алгоритма пропорциональна 2^N. Для $N = 100$ число состояний равно $2^{100} \approx 1.26 \cdot 10^{30}$, что превосходит число атомов во вселенной. Динамика по профилю работает только при $N \leq 16..22$.

⚠️ Ловушка №21: Что будет, если в циклическом буфере забыть `% capacity`?

В чём подвох (где ошибаются): Классический баг выхода за пределы массива.

Эталонный ответ: Индексы `head` и `tail` продолжат монотонно расти и выйдут за границы выделенного массива `A[0..capacity-1]`, что вызовет порчу памяти или Segmentation fault.

⚠️ Ловушка №22: Какова пространственная сложность восстановления ответа в 0-1 рюкзаке?

В чём подвох (где ошибаются): Студенты забывают, что 1D оптимизация не позволяет восстановить ответ.

Эталонный ответ: Для восстановления ответа необходимо знать всю таблицу переходов, то есть требуется хранить полную двумерную матрицу $\mathrm{dp}[n][W]$ памяти $\Theta(nW)$ (или использовать оптимизацию Хиршберга / дерево переходов). Одномерного массива $O(W)$ для восстановления недостаточно!

⚠️ Ловушка №23: Почему слияние в MergeSort нельзя сделать in-place без ухудшения асимптотики?

В чём подвох (где ошибаются): Сложность объединения массивов без буфера.

Эталонный ответ: Чтобы слить два смежных массива in-place без дополнительной памяти, при вставке элемента из правой половины придется сдвигать все оставшиеся элементы левой половины на 1 позицию вправо. Это превратит шаг слияния из $O(n)$ в $O(n^2)$, а всю сортировку — в медленный аналог Insertion Sort.

⚠️ Ловушка №24: Почему сортировки сравнениями не могут работать быстрее $\Omega(n \log n)$?

В чём подвох (где ошибаются): Теорема о дереве решений.

Эталонный ответ: Любая сортировка сравнениями моделируется бинарным деревом решений. Листьями дерева являются все возможные перестановки входа, коих $n!$. Дерево с $n!$ листьями имеет высоту не менее $\log_2(n!) = \sum_{i=1}^{n} \log_2 i \geq (\frac{n}{2}) \log_2(\frac{n}{2}) \in \Omega(n \log n)$.

⚠️ Ловушка №25: Является ли Counting Sort сортировкой сравнениями?

В чём подвох (где ошибаются): Классификация сортировок.

Эталонный ответ: НЕТ. Counting Sort не сравнивает элементы друг с другом ($A[i] < A[j]$). Он использует числовые значения ключей как прямые индексы в массиве счетчиков. Поэтому нижняя граница $\Omega(n \log n)$ к нему не применима.

ЧАСТЬ 4. КОМАНДНЫЙ ТУРНИР: ПЕРЕКРЁСТНЫЙ ОПРОС (15 БАТТЛ-ЗАДАЧ)

⚔️ Командный раунд №1: Медиана числового потока в реальном времени

Кейс-задача: Числа поступают бесконечным потоком по одному. В любой момент времени система должна уметь возвратить текущую медиану за $O(1)$. Предложите структуру данных и оцените время вставки.

Оптимальное решение и обоснование команды: Две сбалансированные кучи:
- Max-Heap для нижней половины чисел ($\leq$ медианы).
- Min-Heap для верхней половины чисел ($\geq$ медианы).
- **Инвариант:** размеры куч отличаются максимум на 1.
- При поступлении числа: добавляем в соответствующую кучу и при необходимости балансируем (переносим корень одной кучи в другую). Вставка: $O(\log n)$.
- Получение медианы: корень большей кучи (или среднее двух корней при четном числе элементов) за $O(1)$!

⚔️ Командный раунд №2: Топ-K самых частых слов в терабайтном лог-файле

Кейс-задача: В лог-файле миллиарды строк. Нужно найти ровно K наиболее часто встречающихся слов ($K = 100$). Объем лога превышает RAM.

Оптимальное решение и обоснование команды:
1. Слова считываются потоком и частоты накапливаются в хеш-таблице (при нехватке RAM — MapReduce или внешняя сортировка с разбиением на партиции).
2. Для выбора Топ-K: заводим **Min-Heap фиксированного размера K**.
3. Проходим по парам `(слово, частота)`. Если текущая частота больше минимума в куче (`heap.top()`), извлекаем корень и вставляем новое слово за $O(\log K)$.
4. Память для выбора: всего $O(K)$, время $O(U \log K)$, где U — число уникальных слов.

⚔️ Командный раунд №3: Сортировка K-отсортированного массива

Кейс-задача: Дан массив размера n, в котором каждый элемент находится на расстоянии не более k от своей истинной позиции в отсортированном массиве ($k \ll n$, например $k = 100, n = 10^7$). Как отсортировать за $O(n \log k)$?

Оптимальное решение и обоснование команды: Min-Heap размера $k + 1$:
1. Помещаем первые $k + 1$ элементов массива в Min-Heap за $O(k)$.
2. Истинный минимальный элемент всего массива гарантированно лежит среди этих $k + 1$ элементов!
3. Извлекаем минимум (`extractMin`), записываем в результирующую позицию 0 массива, и добавляем следующий элемент из входа в кучу.
4. Повторяем для всех n элементов.

- Время: n операций с кучей размера $k + 1 \to O(n \log k)$. Память: $O(k)$.

⚔️ Командный раунд №4: Скользящий максимум за O(n)

Кейс-задача: Дан массив длины n и размер окна k. Найти максимум в каждом окне при сдвиге на 1 вправо за строго линейное время $O(n)$.

Оптимальное решение и обоснование команды: Монотонный дек (Monotonic Deque):
- В деке хранятся индексы элементов в порядке убывания их значений ($A[d_1] > A[d_2] > ...$).
- При добавлении $A[i]$: удаляем с конца дека (`pop_back`) все элементы $\leq A[i]$ (они никогда не станут максимумом).
- Удаляем из начала дека элементы с индексами $\leq i - k$ (вышедшие из окна).
- Максимум текущего окна — элемент в голове дека ($A[d_0]$) за $O(1)$.
- Каждый индекс добавляется и удаляется из дека ровно 1 раз $\to O(n)$ суммарно!

⚔️ Командный раунд №5: Слияние K отсортированных списков

Кейс-задача: Дано k отсортированных связных списков общей длиной N. Как объединить их в один отсортированный список за $O(N \log k)$?

Оптимальное решение и обоснование команды: K-way Merge через Min-Heap:
1. Помещаем головы всех k списков в Min-Heap размера k за $O(k)$.
2. Извлекаем минимальный узел, прикрепляем его к результирующему списку.
3. Если у извлеченного узла есть следующий элемент `next`, добавляем его в кучу за $O(\log k)$.
4. Время: N извлечений и вставок в кучу размера $k \to O(N \log k)$. Память: $O(k)$.

⚔️ Командный раунд №6: Проверка правильности скобочной последовательности

Кейс-задача: Строка длины 10^7 содержит скобки `()[]{}`. Определить корректность за $O(n)$ времени и проверить возможность Stack Overflow.

Оптимальное решение и обоснование команды: Итеративный стек на векторе с предвыделением памяти:
- Используем `std::vector<char>` и вызываем `reserve(n)` для предотвращения перевыделений.
- Сканируем строку: открывающую скобку кладем в стек (`push_back`), для закрывающей проверяем совпадение с `stack.back()` и удаляем (`pop_back`).
- Так как стек живет в куче (heap), а не на системном стеке вызовов (call stack), переполнение стека (Stack Overflow) невозможно. Время $O(n)$, память $O(n)$.

⚔️ Командный раунд №7: Структура данных со вставкой, удалением и getRandom за O(1)

Кейс-задача: Спроектировать контейнер, поддерживающий `insert(val)`, `remove(val)` и `getRandom()` за строго $O(1)$ в среднем.

Оптимальное решение и обоснование команды: Комбинация динамического массива и хеш-таблицы:
- Вектор `vals` хранит сами элементы (для равномерного случайного выбора по индексу `rand() % size`).
- Хеш-таблица `indices` хранит отображение `val -> index_in_vals`.
- `insert(val)`: добавляем в конец вектора, запоминаем индекс в таблице.
- `remove(val)`: находим индекс удаляемого элемента, меняем его местами с **последним** элементом вектора (`swap(vals[idx], vals.back())`), обновляем индекс перемещенного элемента в хеш-таблице и делаем `vals.pop_back()` за $O(1)$!

⚔️ Командный раунд №8: Поиск двух чисел с заданной суммой (Two Sum)

Кейс-задача: Дан несортированный массив n чисел. Найти два числа, сумма которых равна S. Сравнить решения.

Оптимальное решение и обоснование команды:
1. **Через хеш-таблицу:** сканируем массив, для каждого x проверяем наличие $S - x$ в `std::unordered_set`. Время $O(n)$ в среднем, память $O(n)$.
2. **Через сортировку и два указателя:** сортируем массив за $O(n \log n)$, ставим указатели на начало и конец ($L = 0, R = n - 1$). Если $A[L] + A[R] < S \to L++$, если $> S \to R--$. Память $O(1)$ (если сортировка in-place).

⚔️ Командный раунд №9: 0-1 Рюкзак при n = 40 и огромной вместимости W = 10^9

Кейс-задача: Вместимость рюкзака $W = 10^9$, число предметов $n = 40$. Классическое ДП требует $O(nW) = 4 \cdot 10^{10}$ операций и падает по памяти. Как решить задачу?

Оптимальное решение и обоснование команды: Метод «Встреча посередине» (Meet-in-the-middle):
1. Делим 40 предметов на две равные группы по 20 предметов.
2. Для каждой группы перебираем все $2^{20} \approx 10^6$ подмножеств, вычисляя суммарный вес и стоимость.
3. Первую половину оставляем как массив пар `(вес, стоимость)`, удаляя неоптимальные точки и сортируя по весу.
4. Для каждого набора из второй половины с весом w_2 ищем бинарным поиском лучший подходящий набор из первой половины с весом $\leq W - w_2$.
5. Время: $O(2^{\frac{n}{2}} \cdot n) \approx 20 \cdot 10^6$ операций (доли секунды!).

⚔️ Командный раунд №10: Распределенный Quickselect на кластере

Кейс-задача: Массив терабайтного размера разбит на 100 машин. Как найти медиану без пересылки всех данных на один сервер?

Оптимальное решение и обоснование команды:
1. Координатор запрашивает случайный образец (sample) с каждой машины и вычисляет медиану образцов как pivot.
2. Каждая машина параллельно считает число элементов $< \text{pivot}$ и $> \text{pivot}$.
3. Координатор суммирует счетчики. Если позиция pivot совпала с k, медиана найдена. Иначе координатор дает команду отбросить ненужную половину на всех серверах и рекурсивно сузить поиск.

⚔️ Командный раунд №11: Планировщик задач с динамическим изменением приоритета и отменой

Кейс-задача: Очередь задач требует операций: добавить задачу, взять задачу с максимальным приоритетом, повысить приоритет задачи по её UUID и отменить (удалить) задачу по UUID. Все операции за $O(\log n)$.

Оптимальное решение и обоснование команды: Max-Heap с обратной индексацией по хеш-таблице:
- Хеш-таблица хранит `UUID -> index_in_heap`.
- При `swap` элементов в куче синхронно обновляются индексы в хеш-таблице.
- Повышение приоритета: изменяем приоритет и вызываем `siftUp(pos[UUID])`.
- Отмена задачи: заменяем элемент последним в куче и вызываем `siftUp` или `siftDown`. Все операции за $O(\log n)$.

⚔️ Командный раунд №12: Проверка «является ли вершина U предком вершины V» в дереве за O(1)

Кейс-задача: Дано дерево из 10^6 вершин. Поступает 10^6 запросов: «верно ли, что U является предком V?».

Оптимальное решение и обоснование команды: Входные и выходные таймштампы DFS:
1. Запускаем обход DFS, поддерживая глобальный счетчик времени `timer`.
2. При входе в вершину x: `tin[x] = ++timer`. При выходе: `tout[x] = ++timer`.
3. **Критерий предка:** Вершина U является предком вершины V тогда и только тогда, когда отрезок времени для V целиком вложен в отрезок для U: «tin»[U] <= «tin»[V] quad «и» quad «tout»[U] >= «tout»[V].
4. Проверка за строгое $O(1)$!

⚔️ Командный раунд №13: Поиск цикла в односвязном списке без дополнительной памяти

Кейс-задача: Определить, содержит ли односвязный список цикл, используя строго $O(1)$ дополнительной памяти.

Оптимальное решение и обоснование команды: Алгоритм Флойда «Черепаха и Заяц» (Tortoise and Hare):
- Указатель «черепаха» движется на 1 шаг за итерацию (`slow = slow->next`).
- Указатель «заяц» движется на 2 шага за итерацию (`fast = fast->next->next`).
- Если список конечен, заяц дойдет до `nullptr`. Если есть цикл, заяц обязательно догонит черепаху внутри цикла (`slow == fast`). Время $O(n)$, память $O(1)$.

⚔️ Командный раунд №14: Сверхбыстрый расчет N-го числа Фибоначчи при $N = 10^{18}$

Кейс-задача: Вычислить $F_N \bmod 10^9 + 7$ для $N = 10^{18}$. Линейное ДП $O(N)$ невозможно.

Оптимальное решение и обоснование команды: Матричное возведение в степень (Binary Matrix Exponentiation):
- Переход выражается через умножение вектора состояний на матрицу 2×2:

$$\begin{pmatrix} F_{n+1} \\ F_n \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} F_n \\ F_{n-1} \end{pmatrix} \rightarrow \begin{pmatrix} F_{N+1} \\ F_N \end{pmatrix} = \begin{pmatrix} 1 & 1 \\ 1 & 0 \end{pmatrix}^N \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

- Возведение матрицы в степень N выполняется методом бинарного возведения в степень за $O(2^3 \cdot \log_2 N) = O(\log N)$ умножений матриц!
- При $N = 10^{18}$ это занимает всего ~ 60 перемножений матриц 2×2 (микросекунды).

⚔️ Командный раунд №15: Скользящее окно медианы

Кейс-задача: Найти медиану во всех окнах размера k в массиве длины n за $O(n \log k)$.

Оптимальное решение и обоснование команды: Два сбалансированных дерева поиска (или две кучи с ленивым удалением):
- Поддерживаем Max-Heap (нижняя половина) и Min-Heap (верхняя половина), каждое размера $\frac{k}{2}$.
- Для удаления элементов, выходящих из окна: используем **ленивое удаление (Lazy Deletion)** через хеш-таблицу счетчиков удаленных элементов `deleted[val]++`.
- При просмотре корней куч очищаем элементы, помеченные как удаленные.
- Итоговая сложность: $O(n \log k)$ времени и $O(k)$ памяти.